
User Manual

for MPC560XB FEE Driver

Document Number: UM13FEEASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	9
2.2	Overview.....	10
2.3	About this Manual.....	10
2.4	Acronyms and Definitions.....	11
2.5	Reference List.....	11
Chapter 3		
Driver		
3.1	Requirements.....	13
3.2	Driver Design Summary.....	13
3.3	Deviation from Requirements.....	14
3.4	Software specification.....	17
3.4.1	Define Reference.....	17
3.4.2	Macros Reference.....	18
3.4.2.1	Macro FEE_DESERIALIZE.....	18
3.4.2.2	Macro FEE_SERIALIZE.....	18
3.4.3	Enum Reference.....	19
3.4.3.1	Enumeration Fee_BlockStatusType.....	19
3.4.3.2	Enumeration Fee_ClusterStatusType.....	19
3.4.3.3	Enumeration Fee_JobType.....	20
3.4.4	Function Reference.....	21
3.4.4.1	Function Fee_Cancel.....	21
3.4.4.2	Function Fee_EraseImmediateBlock.....	22
3.4.4.3	Function Fee_ForceSwapOnNextWrite.....	24
3.4.4.4	Function Fee_GetJobResult.....	25

Section number	Title	Page
3.4.4.5	Function Fee_GetRunTimeInfo.....	26
3.4.4.6	Function Fee_GetStatus.....	27
3.4.4.7	Function Fee_GetVersionInfo.....	27
3.4.4.8	Function Fee_Init.....	28
3.4.4.9	Function Fee_InvalidateBlock.....	29
3.4.4.10	Function Fee_JobEndNotification.....	30
3.4.4.11	Function Fee_JobErrorNotification.....	30
3.4.4.12	Function Fee_MainFunction.....	31
3.4.4.13	Function Fee_Read.....	32
3.4.4.14	Function Fee_SetMode.....	33
3.4.4.15	Function Fee_Write.....	34
3.4.5	Structs Reference.....	35
3.4.5.1	Structure Fee_BlockConfigType.....	36
3.4.5.2	Structure Fee_BlockInfoType.....	37
3.4.5.3	Structure Fee_ClusterGroupInfoType.....	37
3.4.5.4	Structure Fee_ClusterGroupRuntimeInfoType.....	38
3.4.5.5	Structure Fee_ClusterGroupType.....	39
3.4.5.6	Structure Fee_ClusterType.....	40
3.4.6	Types Reference.....	41
3.4.7	Variables Reference.....	41

Chapter 4 Driver Information

4.1	Fee Data Organization details.....	43
4.2	Memory Dump Example.....	45
4.3	Fee Block Always Available	46
4.4	Managing Cluster and Block Consistency	47
4.5	Cluster Swap.....	47
4.6	Block Update.....	49
4.7	Immediate Block Update.....	50

Section number	Title	Page
Chapter 5		
Tresos Configuration Plug-in		
5.1	Configuration elements of Fee.....	51
5.2	Form IMPLEMENTATION_CONFIG_VARIANT.....	51
5.3	Form FeeGeneral.....	52
5.3.1	FeeDevErrorDetect (FeeGeneral).....	52
5.3.2	FeeIndex (FeeGeneral).....	53
5.3.3	FeeNvmJobEndNotification (FeeGeneral).....	53
5.3.4	FeeNvmJobErrorNotification (FeeGeneral).....	54
5.3.5	FeePollingMode (FeeGeneral).....	54
5.3.6	FeeSetModeSupported (FeeGeneral).....	54
5.3.7	FeeVersionInfoApi (FeeGeneral).....	55
5.3.8	FeeVirtualPageSize (FeeGeneral).....	55
5.3.9	FeeDataBufferSize (FeeGeneral).....	56
5.3.10	FeeBlockAlwaysAvailable (FeeGeneral).....	56
5.4	Form CommonPublishedInformation.....	57
5.4.1	ArReleaseMajorVersion (CommonPublishedInformation).....	58
5.4.2	ArReleaseMinorVersion (CommonPublishedInformation).....	58
5.4.3	ArReleaseRevisionVersion (CommonPublishedInformation).....	59
5.4.4	ModuleId (CommonPublishedInformation).....	59
5.4.5	SwMajorVersion (CommonPublishedInformation).....	60
5.4.6	SwMinorVersion (CommonPublishedInformation).....	60
5.4.7	SwPatchVersion (CommonPublishedInformation).....	61
5.4.8	VendorApiInfix (CommonPublishedInformation).....	61
5.4.9	VendorId (CommonPublishedInformation).....	62
5.5	Form FeePublishedInformation.....	62
5.5.1	FeeBlockOverhead (FeePublishedInformation).....	63
5.5.2	FeeMaximumBlockingTime (FeePublishedInformation).....	63
5.5.3	FeePageOverhead (FeePublishedInformation).....	64

Section number	Title	Page
5.6	Form FeeClusterGroup.....	65
5.6.1	Form FeeCluster.....	65
5.6.1.1	Form FeeSector.....	65
5.6.1.1.1	FeeSectorRef (FeeSector).....	66
5.7	Form FeeBlockConfiguration.....	66
5.7.1	FeeBlockNumber (FeeBlockConfiguration).....	66
5.7.2	FeeBlockSize (FeeBlockConfiguration).....	67
5.7.3	FeeImmediateData (FeeBlockConfiguration).....	67
5.7.4	FeeNumberOfWriteCycles (FeeBlockConfiguration).....	68
5.7.5	FeeClusterGroupRef (FeeBlockConfiguration).....	68
5.7.6	FeeDeviceIndex (FeeBlockConfiguration).....	69

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	15-Feb-2012	Gaetano Stabile	Initial version
1.1	15-May-2013	Rakesh Ranjan	Updated for Bolero RTM 1.0.0
1.2	28-May-2014	Nicolas Regent	Updated for Bolero RTM 1.0.1



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR Fee module (FEE) for MPC560XB .

AUTOSAR FEE driver configuration parameters and deviations from the specification are described in FEE Driver chapter of this document. AUTOSAR FEE driver requirements and APIs are described in the AUTOSAR FEE driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor.

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
-------------------------	--

Table 2-1. MPC5607B Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	--

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Microcontroller Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
ECU	Electronic Control Unit
FLS	Flash memory driver
FEE	Flash EEPROM Emulation
ECC	Error Correcting Code
RAM	Random Access Memory
NVRAM	Non-volatile RAM memory
NvM	NVRAM Manager
BSW	Basic Software

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003FEE Driver Software Specification Document.	V4.0 Rev003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet Reference Manual	MPC5607B_1M03Y - Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet Reference Manual	MPC5604B_2M27V - Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet Reference Manual	MPC5602D_1M18Y - Rev. , 12 DEC 2013

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003 FEE Driver Software Specification document (see the [Reference List](#) table).

3.2 Driver Design Summary

EEPROM (electrically erasable programmable read only memory), which can be byte or word programmed and erased, is often used in automotive electronic control units (ECUs). This flexibility for program and erase operations makes it suitable for data storage of application variables.

For the devices without EEPROM memory, the block-erasable (or sector-erasable) Flash memory can be used to emulate the EEPROM through EEPROM emulation software. The Flash EEPROM emulation module implements emulation of variable-length blocks. Two or more Fee clusters are used to implement such software emulation scheme. The Flash EEPROM Emulation (FEE) provides the upper layer a virtual addressing scheme as well as a "virtually" unlimited number of erase/program cycles. The Flash EEPROM emulation module provides services for reading, writing, erasing and invalidating emulated EEPROM blocks apart from other basic features specified by the software specification.

During the FEE module configuration, each Fee block is assigned to specific Fee cluster group where the Fee block will be physically emulated. Each Fee cluster group consists of at least two Fee clusters, where each Fee cluster consists of at least one Fls logical sector. The list of available Fls logical sectors that can be used by the Fee module for emulation depends on actual Fls driver logical sector list configuration.

The memory operations (read, write etc.) are performed asynchronously. Their respective APIs store actual parameters into internal data structures and immediately return. The *job* is performed by means of state machines, which are driven by repetitive calls to the so called *main functions* of the FEE and FLS drivers (Fee_MainFunction, Fls_MainFunction). These executive functions accomplish their tasks in predefined chunks of data only, and shall be called by the application repeatedly (e.g. periodically in a dedicated task).

Note: For correct Fee operation the underlying Fls driver has to have configured the job-notification callbacks to Fee module:

- FlsJobEndNotification = Fee_JobEndNotification
- FlsJobErrorNotification = Fee_JobErrorNotification

These callbacks control the FEE driver state machine transitions.

3.3 Deviation from Requirements

The driver deviates from the AUTOSAR FEE driver software specification (see the [Reference List](#)) in several details.

There are also some additional requirements (on top of requirements detailed in AUTOSAR FEE Driver software specification) which need to be satisfied for correct operation.

Table [Table 3-1](#) provides Status column description.

Table 3-1. Description of Deviation Status Column

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/R	Unclear Requirement
N/I	Not implemented
N/F	Not fully implemented
I/D	Implemented with Deviations

[Table 3-2](#) identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope of the FEE driver.

Table 3-2. Driver Deviations Table

Requirement	Status	Description	Notes
FEE084	N/R	Imported Type • Fls_AddressType • Fls_LengthType • MemIf_JobResultType • MemIf_ModeType • MemIf_StatusType • Std_ReturnType • Std_VersionInfoType	
FEE013	N/I	The Fee module shall perform inter module checks to avoid integration of incompatible files: all included header files shall be checked by pre-processing directives. The Fee module shall thereby verify that <MODULENAME>_AR_RELEASE_MAJOR_VERSION and <MODULENAME>_AR_RELEASE_MINOR_VERSION are identical to the expected values, where <MODULENAME> is the module short name of the external module, which provides the included header file. If the values are not identical, the Fee module shall raise the development error FEE_E_INCOMPATIBLE_VERSIONS.	
FEE110_Conf	N/I	Number of write cycles required for this block.	
FEE022	N/F	If the current module status is MEMIF_IDLE or if the current module status is MEMIF_BUSY_INTERNAL and the internal management operation can be suspended or aborted, the function Fee_Read shall accept the read request, copy the given / computed parameters to module internal variables, initiate a read job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK.	
FEE172	N/F	If the current module status is MEMIF_UNINIT or MEMIF_BUSY or MEMIF_BUSY_INTERNAL and the internal management operation can't be suspended or aborted, the function Fee_Read shall reject the job request and return with E_NOT_OK.	
FEE025	N/F	If the current module status is MEMIF_IDLE or if the current module status is MEMIF_BUSY_INTERNAL and the internal management operation can be suspended or aborted, the function Fee_Write shall accept the write request, copy the given / computed parameters to module internal variables, initiate a write job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK.	
FEE174	N/F	If the current module status is MEMIF_UNINIT or MEMIF_BUSY or MEMIF_BUSY_INTERNAL and the internal management operation can't be suspended or aborted, the function Fee_Write shall reject the job request and return with E_NOT_OK.	
FEE179	N/F	If the current module status is MEMIF_BUSY_INTERNAL and if the internal management operation can be suspended without jeopardizing the data consistency: the function Fee_MainFunction shall save all information which is necessary to resume the internal management operation,	

Table continues on the next page...

Table 3-2. Driver Deviations Table (continued)

Requirement	Status	Description	Notes
		suspend the internal management operation and start processing the job requested by the upper layer.	
FEE180	N/F	If the current module status is MEMIF_BUSY_INTERNAL and if the internal management operation can be aborted without jeopardizing the data consistency: the function Fee_MainFunction shall save all information which is necessary to restart the internal management operation, abort the internal management operation and start processing the job requested by the upper layer.	
FEE181	N/F	If an internal management operation has been suspended because of a job request from the upper layer, the function Fee_MainFunction shall resume this internal management operation once the job requested by the upper layer has been finished.	
FEE182	N/F	If an internal management operation has been aborted because of a job request from the upper layer, the function Fee_MainFunction shall restart this internal management operation once the job requested by the upper layer has been finished.	
FEE183	N/F	If the write request addresses a block containing immediate data, the function Fee_Write shall accept the write request, even if the current module status is MEMIF_BUSY_INTERNAL and the internal management operation can't be suspended or aborted.	
FEE100	N/R	Only those bits of the 16bit block number, that do not denote a specific dataset or redundant copy shall be used for address calculation.	Unclear implementation of dataset concept.
FEE102	N/I	The configuration of the Fee module shall define the expected number of erase/write cycles for each logical block in the configuration parameter FeeNumberOfWriteCycles.	Not used due to usage of the two or more clusters emulation algorithm.
FEE103	N/I	If the underlying flash device or device driver does not provide at least the configured number of erase/write cycles per physical memory cell, the FEE module shall provide mechanisms to spread the write access such that the physical device is not overstressed. This shall also apply to all management data used internally by the FEE module.	Not used due to usage of the two or more clusters emulation algorithm.
FEE081	N/R	The function Fee_Cancel shall reset the FEE module's internal variables to make the module ready for a new job request.	
FEE034	N/R	If no internal operation is currently ongoing, the function Fee_GetStatus shall call the "GetStatus" function of the underlying flash driver and pass its return value back to the caller.	Misleading requirement, it shall return actual Fee status (which depends on internal Fee states).
FEE035	N/R	The function Fee_GetJobResult shall call the "GetJobResult" function of the underlying flash driver and pass the return value back to the caller.	Misleading requirement, it shall return actual Fee job result.
FEE064	N/R	The function Fee_GetVersionInfo shall return the version information of the FEE module. The version information includes: Module Id, Vendor Id, Vendor specific version numbers (BSW00407)	implementation also fills VersionInfoPtr->instanceID

Table continues on the next page...

Table 3-2. Driver Deviations Table (continued)

Requirement	Status	Description	Notes
FEE082	N/F	If source code for caller and callee of the function Fee_GetVersionInfo is available, the FEE module should realize this function as a macro. The FEE module should define this macro in the module's header file.	The function will be implemented as function, not as a macro.
FEE075	N/F	The function Fee_MainFunction shall check, whether the block requested for reading has been invalidated by the upper layer module. If so, the function Fee_MainFunction shall set the job result to MEMIF_BLOCK_INVALID, call the job error notification function if configured.	MEMIF_BLOCK_INVALID is return value also for not-existing Fee blocks in NVM.
FEE023	N/F	The function Fee_MainFunction shall check the consistency of the logical block being read before notifying the caller. If an inconsistency of the read data is detected, the function Fee_MainFunction shall set the job result to MEMIF_BLOCK_INCONSISTENT and call the error notification routine of the upper layer.	MEMIF_BLOCK_INCONSISTENT is return value also for Fee blocks that has mismatch block-size or immediate-flag.
FEE109	N/R	FeeIndex	Not used by FEE module.
FEE114	N/R	FeePollingMode	Not supported by FEE module, unclear polling concept.
FEE110	N/I	FeeNumberOfWriteCycles	Not supported by FEE module due to emulation scheme.
FEE106	N/R	FeeDeviceIndex	Not used by FEE module.
FEE099	N/R	NvM_JobErrorNotification	
FEE104	N/R	Service to report development errors.	
FEE105	N/R	FLS Api Interface	
FEE007	N/R	Depending on the implementation of the FEE module and the exact address format used, the functions of the FEE module shall combine the 16bit block number and 16bit address offset to derive the physical flash address needed for the underlying flash driver.	Unclear implementation of dataset concept.
FEE114_Conf	N/I	Pre-processor switch to enable and disable the polling mode for this module. true: Polling mode enabled, callback functions (provided to FLS module) disabled. false: Polling mode disabled, callback functions (provided to FLS module) enabled.	
FEE098	N/R	NvM_JobEndNotification	

3.4 Software specification

The following sections contain driver software specifications.

3.4.1 Define Reference

Constants supported by the FEE driver.

3.4.2 Macros Reference

Macros supported by the FEE driver.

3.4.2.1 Macro FEE_DESERIALIZE

Deserialize scalar parameter from the buffer.

Pre: deserialPtr must be valid pointer.

Post: increments the deserialPtr by sizeof(paramType).

Violates: Function-like macro defined

Prototype: FEE_DESERIALIZE(deserialPtr,paramVal,paramType)

**Table 3-3. FEE_DESERIALIZE
Arguments**

Name	Direction	Description
deserialPtr	input, output	Pointer to source buffer.
paramVal	output	Deserialized parameter.
paramType	input	Type of serialized parameter.

3.4.2.2 Macro FEE_SERIALIZE

Serialize scalar parameter into the buffer.

Pre: serialPtr must be valid pointer.

Post: increments the serialPtr by sizeof(paramType).

Violates: Function-like macro defined

Prototype: FEE_SERIALIZE(paramVal,paramType,serialPtr)

Table 3-4. FEE_SERIALIZE Arguments

Name	Direction	Description
paramVal	input	Serialized parameter.
paramType	input	Type of serialized parameter.
serialPtr	input, output	Pointer to target buffer.

3.4.3 Enum Reference

Enumeration of all constants supported by the FEE driver.

3.4.3.1 Enumeration Fee_BlockStatusType

Status of Fee block header.

Implements: DFEE03020

Table 3-5. Enumeration Fee_BlockStatusType Values

Name	Initializer	Description
FEE_BLOCK_VALID	0	Fee block is valid.
FEE_BLOCK_INVALID		Fee block is invalid (has been invalidated).
FEE_BLOCK_INCONSISTENT		Fee block is inconsistent (contains bogus data).
FEE_BLOCK_HEADER_INVALID		Fee block header is garbled.
FEE_BLOCK_INVALIDATED		Fee block header is invalidated by Fee_InvalidateBlock(BlockNumber)(not used when FEE_BLOCK_ALWAYS_AVAILABLE == STD_OFF).
FEE_BLOCK_HEADER_BLANK		Fee block header is blank (end of Fee block header list).

3.4.3.2 Enumeration Fee_ClusterStatusType

Status of Fee cluster header.

Implements: DFEE03021

Table 3-6. Enumeration Fee_ClusterStatusType Values

Name	Initializer	Description
FEE_CLUSTER_VALID	0	Fee cluster is valid.
FEE_CLUSTER_INVALID		Fee cluster is invalid.
FEE_CLUSTER_INCONSISTENT		Fee cluster is inconsistent (contains bogus data).
FEE_CLUSTER_HEADER_INVALID		Fee cluster header is garbled.

3.4.3.3 Enumeration Fee_JobType

Type of job currently executed by Fee_MainFunction.

Implements: DFEE03022

Table 3-7. Enumeration Fee_JobType Values

Name	Initializer	Description
FEE_JOB_READ	0	Read Fee block.
FEE_JOB_WRITE		Write Fee block to flash.
FEE_JOB_WRITE_DATA		Write Fee block data to flash.
FEE_JOB_WRITE_UNALIGNED_DATA		Write unaligned rest of Fee block data to flash.
FEE_JOB_WRITE_VALIDATE		Validate Fee block by writing validation flag to flash.
FEE_JOB_WRITE_DONE		Finalize validation of Fee block.
FEE_JOB_INVALID_BLOCK		Invalidate Fee block by writing the invalidation flag to flash.
FEE_JOB_INVALID_BLOCK_DONE		Finalize invalidation of Fee block.
FEE_JOB_ERASE_IMMEDIATE		Erase (pre-allocate) immediate Fee block.
FEE_JOB_ERASE_IMMEDIATE_DONE		Finalize erase (pre-allocation) of Fee block.
FEE_JOB_INT_SCAN		Initialize the cluster scan job.
FEE_JOB_INT_SCAN_CLR_HDR_PARSE		Parse Fee cluster header.

Table continues on the next page...

Table 3-7. Enumeration Fee_JobType Values (continued)

Name	Initializer	Description
FEE_JOB_INT_SCAN_CLR		Scan active cluster of current cluster group.
FEE_JOB_INT_SCAN_CLR_FMT		Format first Fee cluster.
FEE_JOB_INT_SCAN_CLR_FMT_DONE		Finalize format of first Fee cluster.
FEE_JOB_INT_SCAN_BLOCK_HDR_PARSE		Parse Fee block header.
FEE_JOB_INT_SWAP_BLOCK		Copy next block from source to target cluster.
FEE_JOB_INT_SWAP_CLR_FMT		Format current Fee cluster in current Fee cluster group.
FEE_JOB_INT_SWAP_DATA_READ		Read data from source cluster to internal Fee buffer.
FEE_JOB_INT_SWAP_DATA_WRITE		Write data from internal Fee buffer to target cluster.
FEE_JOB_INT_SWAP_CLR_VLD_DONE		Finalize cluster validation.
FEE_JOB_DONE		No more subsequent jobs to schedule.

3.4.4 Function Reference

Functions supported by the FEE driver.

3.4.4.1 Function Fee_Cancel

Service to call the cancel function of the underlying flash driver.

Details:

The function Fee_Cancel and the cancel function of the underlying flash driver are asynchronous w.r.t. an ongoing read, erase or write job in the flash memory.

Pre: The module must be initialized.

Post: Changes Fee_ModuleStatus module status and job result Fee_JobResult internal variables.

Note

The function Autosar Service ID[hex]:
0x04.Asynchronous.Non Reentrant.

The function Fee_Cancel and the cancel function of the underlying flash driver are asynchronous w.r.t. an ongoing read, erase or write job in the flash memory.

Implements: DFEE03012

Pre: The module must be initialized.

Post: Changes Fee_ModuleStatus module status and job result Fee_JobResult internal variables.

Implements: DFEE03012

Prototype: void Fee_Cancel(void);

3.4.4.2 Function Fee_EraseImmediateBlock

Service to erase a logical block.

Details:

Return the version information of the Fee module.

The function Fee_EraseImmediateBlock shall take the block number and calculate the corresponding memory block address. The function Fee_EraseImmediateBlock shall ensure that the FEE module can write immediate data. Whether this involves physically erasing a memory area and therefore calling the erase function of the underlying driver depends on the implementation. If development error detection for the FEE module is enabled, the function Fee_EraseImmediateBlock shall check whether the addressed logical block is configured as containing immediate data (configuration parameter FeeImmediateData == TRUE). If not, the function Fee_EraseImmediateBlock shall report the error code FEE_E_INVALID_BLOCK_NO.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, BlockNumber must be valid, and type of Fee block must be immediate.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_Job, and Fee_JobResult job control internal variables.

The version information includes: Module Id, Vendor Id, Vendor specific version numbers

Implements: DFEE03017

Note

The function Autosar Service ID[hex]:
0x09.Asynchronous.Non Reentrant.

Pre: VersionInfoPtr must not be NULL_PTR.

The function Fee_EraseImmediateBlock shall take the block number and calculate the corresponding memory block address. The function Fee_EraseImmediateBlock shall ensure that the FEE module can write immediate data. Whether this involves physically erasing a memory area and therefore calling the erase function of the underlying driver depends on the implementation. If development error detection for the FEE module is enabled, the function Fee_EraseImmediateBlock shall check whether the addressed logical block is configured as containing immediate data (configuration parameter FeeImmediate- Data == TRUE). If not, the function Fee_EraseImmediateBlock shall report the error code FEE_E_INVALID_BLOCK_NO.

Implements: DFEE03016

Note

The function Autosar Service ID[hex]: 0x08.SynchronousNon
Reentrant

Implements: DFEE14001 Service to erase a logical block.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, BlockNumber must be valid, and type of Fee block must be immediate.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_Job, and Fee_JobResult job control internal variables.

Implements: DFEE03017

Note

The function Autosar Service ID[hex]: 0x09.AsynchronousNon
Reentrant

Prototype: Std_ReturnType Fee_EraseImmediateBlock(uint16 BlockNumber);

Table 3-8. Fee_EraseImmediateBlock Arguments

Type	Name	Direction	Description
uint16	BlockNumber	input	Number of logical block, also denoting.
VersionInfoPtr	output	Pointer to where to store the version	

Table continues on the next page...

Table 3-8. Fee_EraseImmediateBlock Arguments (continued)

Type	Name	Direction	Description
		information of this module.	
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in emulated EEPROM.

Table 3-9. Fee_EraseImmediateBlock Return Values

Name	Description
E_OK	The job was accepted by the underlying memory driver.
E_NOT_OK	The job has not been accepted by the underlying memory driver. start address of that block in emulated EEPROM.
E_OK	The job was accepted by the underlying memory driver.
E_NOT_OK	The job has not been accepted by the underlying memory driver.

3.4.4.3 Function Fee_ForceSwapOnNextWrite

Service to prepare the driver for a cluster swap in the selected cluster group.

Details:

While the computed amount of memory is allocated as a result of Fee_Write call for plain data blocks, for immediate data blocks memory gets completely pre-allocated through Fee_EraseImmediateBlock function (i.e. Fee_Write does not change the remaining space). As a result, swaps triggered by the planned Fee_ForceSwapOnNextWrite function behave the same way, or in other words, an operation that really activates the physical swap must be either Fee_Write on plain FEE block or Fee_EraseImmediateBlock on immediate data block.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy and clrGrpIndex must be valid.

Implements: DFEE03084

Note

As this API manipulates the internal driver state, it has to be claimed non-reentrant and colliding with other FEE ASR APIs

Prototype: Std_ReturnType Fee_ForceSwapOnNextWrite(uint8 clrGrpIndex);

Table 3-10. Fee_ForceSwapOnNextWrite Arguments

Type	Name	Direction	Description
uint8	clrGrpIndex	input	Index of the selected cluster group.

Table 3-11. Fee_ForceSwapOnNextWrite Return Values

Name	Description
E_NOT_OKmodule	Is not initialized, busy or clrGrpIndex is not in the valid range.
E_OKNo	More space available in the selected cluster.

3.4.4.4 Function Fee_GetJobResult

Return the result of the last job.

Details:

Return the result of the last job synchronously.

Return: MemIf_JobResultType.

Return the result of the last job synchronously.

Implements: DFEE03014

Note

The function Autosar Service ID[hex]: 0x06.Synchronous.Non Reentrant.

Return: MemIf_JobResultType.

Implements: DFEE03014

Note

The function Autosar Service ID[hex]: 0x06.SynchronousNon Reentrant

Prototype: MemIf_JobResultType Fee_GetJobResult(void);

Table 3-12. Fee_GetJobResult Return Values

Name	Description
MEMIF_JOB_OK	The job has been finished successfully.
MEMIF_JOB_FAILED	The job has not been finished successfully.

Table continues on the next page...

Table 3-12. Fee_GetJobResult Return Values (continued)

Name	Description
MEMIF_JOB_PENDING	The job has not yet been finished.
MEMIF_JOB_CANCELED	The job has been canceled.
MEMIF_BLOCK_INCONSISTENT	The requested block is inconsistent, it may contain corrupted data.
MEMIF_BLOCK_INVALID	The requested block has been invalidated, the requested read operation can not be performed.
MEMIF_JOB_OK	The job has been finished successfully.
MEMIF_JOB_FAILED	The job has not been finished successfully.
MEMIF_JOB_PENDING	The job has not yet been finished.
MEMIF_JOB_CANCELED	The job has been canceled.
MEMIF_BLOCK_INCONSISTENT	The requested block is inconsistent, it may contain corrupted data.
MEMIF_BLOCK_INVALID	

3.4.4.5 Function Fee_GetRunTimeInfo

Service to read runtime information in the selected cluster.

Return: Fee_ClusterGroupRuntimeInfoType.

Pre: The module must be initialized, not busy and clrGrpIndex must be valid.

Implements: DFEE03083

Prototype: Fee_ClusterGroupRuntimeInfoType Fee_GetRunTimeInfo(uint8 clrGrpIndex);

Table 3-13. Fee_GetRunTimeInfo Arguments

Type	Name	Direction	Description
uint8	clrGrpIndex	input	Index of the selected cluster group.

Table 3-14. Fee_GetRunTimeInfo Return Values

Name	Description
current	Cluster total size.
current	Cluster available space.
the	Block header overhead (blk header, valid flag and inval flag),.
the	Virtual page size (the size of an allocation unit).
number	Of cluster swap already performed.

3.4.4.6 Function Fee_GetStatus

Return the Fee module state.

Details:

Return the Fee module state synchronously.

Note

The function Autosar Service ID[hex]: 0x05.SynchronousNonReentrant

Return: Fee_ModuleStatus.

Return the Fee module state synchronously.

Implements: DFEE03013

Return: Fee_ModuleStatus.

Implements: DFEE03013

Note

The function Autosar Service ID[hex]: 0x05.SynchronousNonReentrant

Prototype: MemIf_StatusType Fee_GetStatus(void);

Table 3-15. Fee_GetStatus Return Values

Name	Description
MEMIF_UNINIT	Module has not been initialized (yet).
MEMIF_IDLE	Module is currently idle.
MEMIF_BUSY	Module is currently busy.
MEMIF_BUSY_INTERNAL	Module is busy with internal management operations.
MEMIF_UNINIT	Module has not been initialized (yet).
MEMIF_IDLE	Module is currently idle.
MEMIF_BUSY	Module is currently busy.
MEMIF_BUSY_INTERNAL	Module is busy with internal management operations.

3.4.4.7 Function Fee_GetVersionInfo

Return the version information of the Fee module.

Details:

The version information includes: Module Id, Vendor Id, Vendor specific version numbers.

Pre: VersionInfoPtr must not be NULL_PTR.

Implements: DFEE03016

Note

The function Autosar Service ID[hex]: 0x08.Synchronous.Non Reentrant.

Prototype: void Fee_GetVersionInfo(Std_VersionInfoType *VersionInfoPtr);

Table 3-16. Fee_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	VersionInfoPtr	output	Pointer to where to store the version information of this module .

3.4.4.8 Function Fee_Init

Service to initialize the FEE module.

Details:

The function Fee_Init shall initialize the Flash EEPROM Emulation module.

Pre: The FEE module's environment shall not call the function Fee_Init shall during a running operation of the FEE module.

Note

The function Autosar Service ID[hex]: 0x00.SynchronousNon Reentrant

The function Fee_Init shall initialize the Flash EEPROM Emulation module.

Implements: DFEE03008

Pre: The FEE module's environment shall not call the function Fee_Init shall during a running operation of the FEE module.

Implements: DFEE03008

Note

The function Autosar Service ID[hex]: 0x00.SynchronousNonReentrant

Prototype: `void Fee_Init(void);`

3.4.4.9 Function Fee_InvalidateBlock

Service to invalidate a logical block.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, and BlockNumber must be valid.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_Job, and Fee_JobResult job control internal variables. EEPROM.

Implements: DFEE03015

Note

The function Autosar Service ID[hex]: 0x07.Asynchronous.Non Reentrant.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, and BlockNumber must be valid.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_Job, and Fee_JobResult job control internal variables.

Implements: DFEE03015

Note

The function Autosar Service ID[hex]: 0x07.AsynchronousNonReentrant

Prototype: `Std_ReturnType Fee_InvalidateBlock(uint16 BlockNumber);`

Table 3-17. Fee_InvalidateBlock Arguments

Type	Name	Direction	Description
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in flash memory.
uint16	BlockNumber	input	Number of logical block, also denoting vstart address of that block in emulated EEPROM.

Table 3-18. Fee_InvalidateBlock Return Values

Name	Description
E_OK	The job was accepted by the underlying memory driver.
E_NOT_OK	The job has not been accepted by the underlying memory driver.
E_OK	The job was accepted by the underlying memory driver.
E_NOT_OK	The job has not been accepted by the underlying memory driver.

3.4.4.10 Function Fee_JobEndNotification

Service to report the FEE module the successful end of an asynchronous operation.

Details:

The underlying flash driver shall call the function Fee_JobEndNotification to report the successful end of an asynchronous operation.

Pre: The module must be initialized.

Post: Changes Fee_ModuleStatus module status and Fee_JobResult internal variables.

Implements: DFEE03006

Note

The function Autosar Service ID[hex]: 0x10.Synchronous.Non
Reentrant

Prototype: void Fee_JobEndNotification(void);

3.4.4.11 Function Fee_JobErrorNotification

Service to report the FEE module the failure of an asynchronous operation.

Details:

The underlying flash driver shall call the function Fee_JobErrorNotification to report the failure of an asynchronous operation.

Pre: The module must be initialized.

Post: Changes Fee_ModuleStatus module status and Fee_JobResult internal variables.

Implements: DFEE03007

Note

The function Autosar Service ID[hex]: 0x11.Synchronous.Non Reentrant.

Prototype: void Fee_JobErrorNotification(void);

3.4.4.12 Function Fee_MainFunction

Service to handle the requested read / write / erase jobs respectively the internal management operations.

Details:

The function shall asynchronously handle the requested read / write / erase jobs respectively the internal management operations. The function shall check, whether the block requested for reading has been invalidated by the upper layer module. If so, the function shall set the job result to MEMIF_BLOCK_INVALID and call the error notification routine of the upper layer if configured. The function shall check the consistency of the logical block being read before notifying the caller. If an inconsistency of the read data is detected, the function shall set the job result to MEMIF_BLOCK_INCONSISTENT and call the error notification routine of the upper layer.

Pre: The module must be initialized.

Note

The function Autosar Service ID[hex]: 0x12.

The function shall asynchronously handle the requested read / write / erase jobs respectively the internal management operations. The function shall check, whether the block requested for reading has been invalidated by the upper layer module. If so, the function shall set the job result to MEMIF_BLOCK_INVALID, call the job error notification function if configured. The function shall check the consistency of the logical block being read before notifying the caller. If an inconsistency of the read data is Specification of FLASH EEPROM Emulation detected, the function shall set the job result to MEMIF_BLOCK_INCONSISTENT and call the error notification routine of the upper layer.

Implements: DFEE03018

Pre: The module must be initialized.

Implements: DFEE03018

Note

The function Autosar Service ID[hex]: 0x12.

Prototype: void Fee_MainFunction(void);

3.4.4.13 Function Fee_Read

Service to initiate a read job.

Details:

The function Fee_Read shall take the block start address and offset and calculate the corresponding memory read address.

Pre: The module must be initialized, not busy, BlockNumber must be valid, Length != 0, DataBufferPtr != NULL_PTR, BlockOffset and (BlockOffset + Length - 1) must be in range.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockOffset, Fee_JobBlockLength, Fee_JobBlockIndex, Fee_JobDataDestPtr, Fee_Job, Fee_JobResult job control internal variables.

Return: Std_ReturnType.

The function Fee_Read shall take the block start address and offset and calculate the corresponding memory read address.

Implements: DFEE03010

Note

The function Autosar Service ID[hex]:
0x02.Asynchronous.Non Reentrant.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, BlockNumber must be valid, Length != 0, DataBufferPtr != NULL_PTR, BlockOffset and (BlockOffset + Length - 1) must be in range.

Post: changes Fee_ModuleStatus module status and Fee_JobBlockOffset, Fee_JobBlockLength, Fee_JobBlockIndex, Fee_JobDataDestPtr, Fee_Job, Fee_JobResult job control internal variables.

Implements: DFEE03010

Note

The function Autosar Service ID[hex]: 0x02.AsynchronousNonReentrant

Prototype: Std_ReturnType Fee_Read(uint16 BlockNumber, uint16 BlockOffset, uint8 *DataBufferPtr, uint16 Length);

Table 3-19. Fee_Read Arguments

Type	Name	Direction	Description
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in flash memory.
uint16	BlockOffset	input	Read address offset inside the block.
uint8 *	DataBufferPtr	output	Pointer to data buffer.
uint16	Length	input	Number of bytes to read.
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in flash memory.
uint16	BlockOffset	input	Read address offset inside the block.
uint8 *	DataBufferPtr	output	Pointer to data buffer.
uint16	Length	input	Number of bytes to read.

Table 3-20. Fee_Read Return Values

Name	Description
E_OK	The read job was accepted by the underlying memory driver.
E_NOT_OK	The read job has not been accepted by the underlying memory driver.
E_OK	The read job was accepted by the underlying memory driver.
E_NOT_OK	The read job has not been accepted by the underlying memory driver.

3.4.4.14 Function Fee_SetMode

Set the Fee module's operation mode to the given Mode.

Details:

Call the Fls_SetMode function of the underlying flash driver.

Pre: The module must be initialized and not busy.

Implements: DFEE03009

Note

The function Autosar Service ID[hex]: 0x01.SynchronousNon Reentrant

Prototype: void Fee_SetMode(MemIf_ModeType Mode);

Table 3-21. Fee_SetMode Arguments

Type	Name	Direction	Description
MemIf_ModeType	Mode	input	(Either MEMIF_MODE_FAST or MEMIF_MODE_SLOW).

3.4.4.15 Function Fee_Write

Service to initiate a write job.

Details:

The function Fee_Write shall take the block start address and calculate the corresponding memory write address. The block address offset shall be fixed to zero. The function Fee_Write shall copy the given / computed parameters to module internal variables, initiate a write job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The FEE module shall execute the write job of the function Fee_Write asynchronously within the FEE module's main function.

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, BlockNumber must be valid, and DataBufferPtr != NULL_PTR. Before call the function "Fee_Write" for immediate date must be called the function "Fee_EraseImmediateBlock".

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_JobDataDestPtr, Fee_Job, Fee_JobResult job control internal variables.

Note

The function Autosar Service ID[hex]:
0x03.Asynchronous.Non Reentrant.

The function Fee_Write shall take the block start address and calculate the corresponding memory write address. The block address offset shall be fixed to zero. The function Fee_Write shall copy the given / computed parameters to module internal variables,

initiate a write job, set the FEE module status to MEMIF_BUSY, set the job result to MEMIF_JOB_PENDING and return with E_OK. The FEE module shall execute the write job of the function Fee_Write asynchronously within the FEE module's main function.

Implements: DFEE03011

Return: Std_ReturnType.

Pre: The module must be initialized, not busy, BlockNumber must be valid, and DataBufferPtr != NULL_PTR. Before call the function "Fee_Write" for immediate date must be called the function "Fee_EraseImmediateBlock".

Post: changes Fee_ModuleStatus module status and Fee_JobBlockIndex, Fee_JobDataDestPtr, Fee_Job, Fee_JobResult job control internal variables.

Implements: DFEE03011

Note

The function Autosar Service ID[hex]: 0x03.AsynchronousNonReentrant

Prototype: Std_ReturnType Fee_Write(uint16 BlockNumber, uint8 *DataBufferPtr);

Table 3-22. Fee_Write Arguments

Type	Name	Direction	Description
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in emulated EEPROM.
uint8 *	DataBufferPtr	output	Pointer to data buffer.
uint16	BlockNumber	input	Number of logical block, also denoting start address of that block in emulated EEPROM.
uint8 *	DataBufferPtr	output	Pointer to data buffer.

Table 3-23. Fee_Write Return Values

Name	Description
E_OK	The write job was accepted by the underlying memory driver.
E_NOT_OK	The write job has not been accepted by the underlying memory driver.
E_OK	The write job was accepted by the underlying memory driver.
E_NOT_OK	The write job has not been accepted by the underlying memory driver.

3.4.5 Structs Reference

Data structures supported by the FEE driver.

3.4.5.1 Structure Fee_BlockConfigType

Fee block configuration structure.

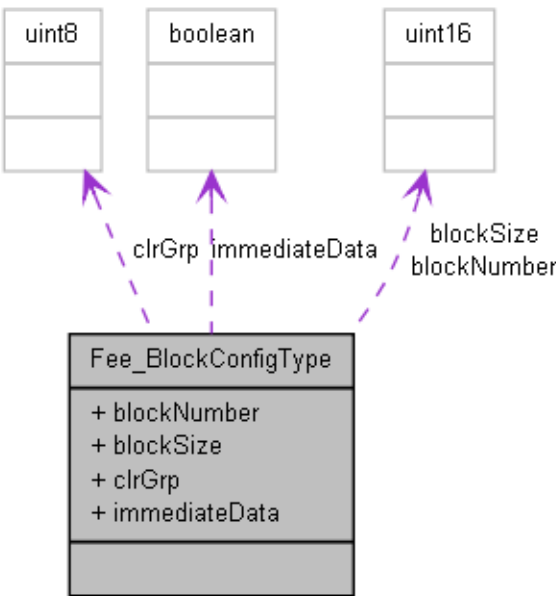


Figure 3-1. Struct Fee_BlockConfigType

Implements: DFEE03003

Declaration:

```
typedef struct
{
    uint16 blockNumber,
    uint16 blockSize,
    uint8 clrGrp,
    boolean immediateData
} Fee_BlockConfigType;
```

Table 3-24. Structure Fee_BlockConfigType member description

Member	Description
blockNumber	Fee block number.
blockSize	Size of Fee block in bytes.
clrGrp	Index of cluster group the Fee block belongs to.
immediateData	TRUE if immediate data block.

3.4.5.2 Structure Fee_BlockInfoType

Fee block run-time status.

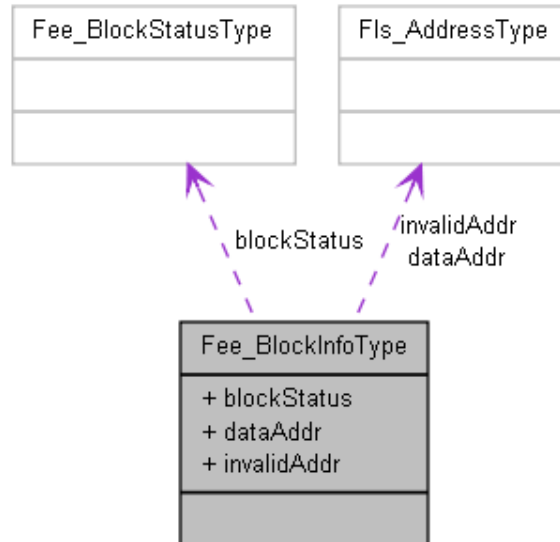


Figure 3-2. Struct Fee_BlockInfoType

Implements: DFEE03024

Declaration:

```
typedef struct
{
    Fee_BlockStatusType blockStatus,
    Fls_AddressType dataAddr,
    Fls_AddressType invalidAddr
} Fee_BlockInfoType;
```

Table 3-25. Structure Fee_BlockInfoType member description

Member	Description
blockStatus	Current status of Fee block.
dataAddr	Address of Fee block data in flash.
invalidAddr	Address of Fee block invalidation field in flash.

3.4.5.3 Structure Fee_ClusterGroupInfoType

Fee cluster group run-time status.

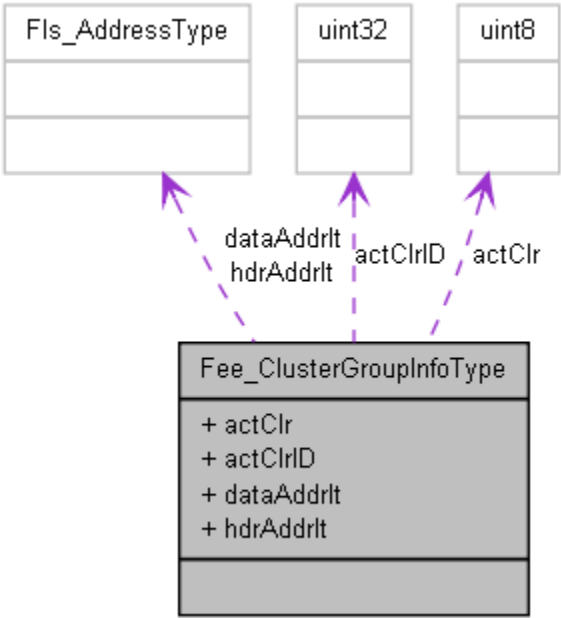


Figure 3-3. Struct Fee_ClusterGroupInfoType

Implements: DFEE03023

Declaration:

```
typedef struct
{
    uint8 actClr,
        uint32 actClrID,
        Fls_AddressType dataAddrIt,
        Fls_AddressType hdrAddrIt
} Fee_ClusterGroupInfoType;
```

Table 3-26. Structure Fee_ClusterGroupInfoType member description

Member	Description
actClr	Index of active cluster.
actClrID	ID of active cluster.
dataAddrIt	Address of current Fee data block in flash.
hdrAddrIt	Address of current Fee block header in flash.

3.4.5.4 Structure Fee_ClusterGroupRuntimeInfoType

Fee cluster group run-time Information.

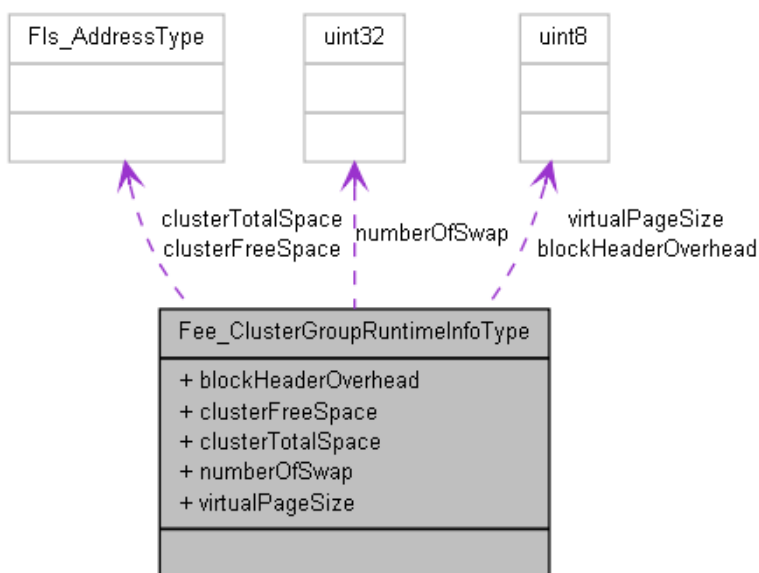


Figure 3-4. Struct Fee_ClusterGroupRuntimeInfoType

Implements: DFEE03085**Declaration:**

```

typedef struct
{
    uint8 blockHeaderOverhead,
        Fls_AddressType clusterFreeSpace,
        Fls_AddressType clusterTotalSpace,
        uint32 numberOfSwap,
        uint8 virtualPageSize
} Fee_ClusterGroupRuntimeInfoType;
  
```

Table 3-27. Structure Fee_ClusterGroupRuntimeInfoType member description

Member	Description
blockHeaderOverhead	Block Overhead (header valid and inval flag).
clusterFreeSpace	Free space in the selected cluster group.
clusterTotalSpace	Total space in the selected cluster group.
numberOfSwap	Number of cluster swap performed in the selected cluster group.
virtualPageSize	Fee Virtual Page Size.

3.4.5.5 Structure Fee_ClusterGroupType

Fee cluster group configuration structure.

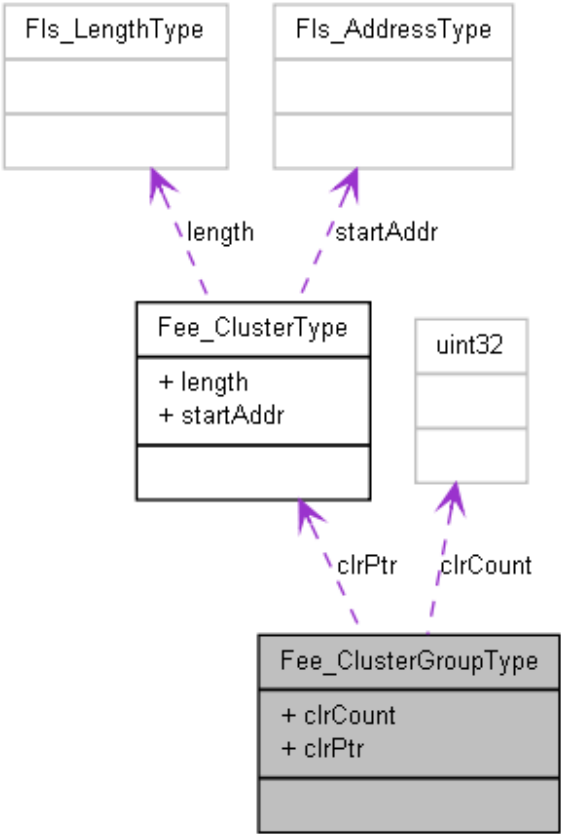


Figure 3-5. Struct Fee_ClusterGroupType

Implements: DFEE03005

Declaration:

```
typedef struct
{
    uint32 clrCount,
    constFee_ClusterType*const clrPtr
} Fee_ClusterGroupType;
```

Table 3-28. Structure Fee_ClusterGroupType member description

Member	Description
clrCount	Pointer to array of Fee cluster configurations.
clrPtr	

3.4.5.6 Structure Fee_ClusterType

Fee cluster configuration structure.

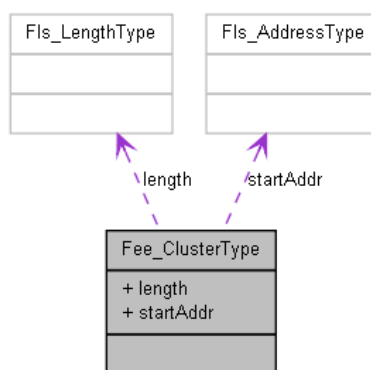


Figure 3-6. Struct Fee_ClusterType

Implements: DFEE03004

Declaration:

```

typedef struct
{
    Fls_LengthType length,
    Fls_AddressType startAddr
} Fee_ClusterType;
  
```

Table 3-29. Structure Fee_ClusterType member description

Member	Description
length	Size of Fee cluster in bytes.
startAddr	Address of Fee cluster in flash.

3.4.6 Types Reference

Types structures supported by the FEE driver.

3.4.7 Variables Reference

Variables supported by the FEE driver.

Chapter 4

Driver Information

4.1 Fee Data Organization details

The FEE module provides upper layers with a 32bit virtual linear address space and uniform segmentation scheme.

This virtual 32bit address shall consist of:

- a 16bit block number - allowing a (theoretical) number of 65536 logical blocks
- a 16bit block offset - allowing a (theoretical) block size of 64KByte per block

The 16bit block number represents a configurable (virtual) paging mechanism.

The organization of flash area reserved for Fee driver is described here below.

The memory area is organized in:

- **Cluster Group** : A group is made by at least 2 Clusters
- **Cluster** : One or more Flash physical sectors containing FEE blocks
- **Block** : Area of Flash containing application data

More clusters could be present in the area but just one is active and contains valid data while the others are not used.

Note: In the example below Fee_VirtualPageSize is set to 8. Header valid flag and invalid flag are aligned each to Fee_VirtualPageSize

Each cluster/block has:

- an header
- data

Table 4-1. Data Organization details

128 bits					
4 bytes		4 bytes	4 bytes	4 bytes	Description
CnrID		Start address	Cluster size	Checksum	Cluster header
Valid flag		not used	not used	not used	Cluster status
Block id	Length	Target address	Checksum	not used	Block 1 header
Valid flag		not used	Invalid flag	not used	Block 1 status
Block id	Length	Target address	Checksum	not used	Block 2 header
Valid flag		not used	Invalid flag	not used	Block 2 status
...					
...					
Block id	Length	Target address	Checksum	not used	Block n-1 header
Valid flag		not used	Invalid flag	not used	Block n-1 status
Block id	Length	Target address	Checksum	not used	Block n header
Valid flag		not used	Invalid flag	not used	Block n status
(padding)					16 byte
(padding)					16 byte
BLOCK n DATA					Block n Data
BLOCK n DATA					Block n Data
BLOCK n DATA					Block n Data
BLOCK n DATA					Block n Data
BLOCK n-1 DATA					Block n-1 Data
BLOCK n-1 DATA					Block n-1 Data
...					
...					
BLOCK 2 DATA					Block 2 Data
BLOCK 2 DATA					Block 2 Data
BLOCK 1 DATA					Block 1 Data
BLOCK 1 DATA					Block 1 Data
BLOCK 1 DATA					Block 1 Data
BLOCK 1 DATA					Block 1 Data

Table 4-2. ClusterHdr Type

uint32 (4 bytes)	uint32 (4 bytes)	uint32 (4 bytes)	uint32 (4 bytes)
CnrID	StartAddress	ClusterSize	checkSum
valFlag	blank1	invalFlag	blank2

- **ClrID:** (uint32) Integer number uniquely identifying the cluster. The number is incremented whenever a new cluster becomes active, i.e. the cluster with highest ClrID is the active one.
- **StartAddress:** (uint32) Configuration data: Start address of the cluster (logical start address of the first flash sector belonging to this cluster).
- **ClusterSize:** (uint32) Configuration data: Length of the cluster.
- **checksum:** (uint32) Sum of the ClrID, StartAddress and ClusterSize fields.
- **val Flag:** (uint32) 0x81 for a valid cluster
- **invalFlag:** (uint32) not-used

Table 4-3. BlockHdr Type

uint32 (4 bytes)	uint32 (4 bytes)	uint32 (4 bytes)	uint32 (4 bytes)
BlockNumber:Length	TargetAddress	checksum	blank1
valFlag	blank2	invalFlag	blank3

- **BlockNumber:** (uint16) Configuration data: Integer number uniquely identifying the block.
- **Length:** (uint16) Configuration data: Length of the block.
- **TargetAddress:** (uint32) Logical address of the beginning of data area of this block.
- **checksum:** (uint32) Sum of the BlockNumber, Length and TargetAddress fields.
- **valFlag:** (uint32) 0x81 for a valid block.
- **invalFlag:** (uint32) Value 0x18 in this field indicates that the block was invalidated.

4.2 Memory Dump Example

The table below shows an example of the cluster dump:

- One group of two clusters is configured.
- The first cluster has start address 0x10000.
- The second cluster has start address 0x18000.
- Two blocks are written.
- The length of the first block is 4.
- The length of the second block is 64.

Table 4-4. Dump Memory example (Fee_VirtualPageSize = 8)

Offset (hex)	4 bytes (hex)	4 bytes (hex)	4 bytes (hex)	4 bytes (hex)	Description
--------------	---------------	---------------	---------------	---------------	-------------

Table continues on the next page...

Table 4-4. Dump Memory example (Fee_VirtualPageSize = 8) (continued)

0000	00	00	00	01	00	00	00	00	00	01	00	00	00	01	00	01	Clr Hdr
0010	81	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	Clr Sts
0020	00	01	00	04	00	00	FF	F8	00	01	FF	FD	FF	FF	FF	FF	Blk1 Hdr
0030	81	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	Blk1 Sts
0040	00	02	00	40	00	00	FF	B8	00	01	FF	FA	FF	FF	FF	FF	Blk2 Hdr
0050	81	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	Blk2 Sts
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	
FFB0	FF	FF	FF	FF	FF	FF	FF	FF	01	01	01	01	01	01	01	01	Blk2 Data
FFC0	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	Blk1 Data
FFD0	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	
FFE0	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	
FFF0	01	01	01	01	01	01	01	01	00	00	00	00	FF	FF	FF	FF	

Note: Values 0x00 and 0x01 represent example application data stored in blocks 1 and 2 respectively, and 0xFF is the value of unprogrammed (erased, blank) flash memory byte.

4.3 Fee Block Always Available

To be consistent with Autosar Requirement **FEE153** (*When a block write operation is started, the FEE module shall mark the corresponding block as inconsistent.*) and **FEE154** (*Upon the successful end of the block write operation, the block shall be marked as consistent (again)*) in case of reset, power loss etc. occur between the writing of the first part of the header (including the checksum) and the writing of the valid flag of the header, neither newly nor previously written data is available.

This is the default behaviour of the driver if "Fee Block Always Available" configuration parameter is set to FALSE



Figure 4-1. FEE153 and FEE154 respected

This behaviour can be modified (thus losing compliance with Autosar requirement FEE153 and FEE154) and if a previous valid instance of the block exists, it is always possible to recover it.



Figure 4-2. FEE153 and FEE154 violated

4.4 Managing Cluster and Block Consistency

The FEE module shall manage the consistency of block when catastrophic events occurs:

If the reset event occurs, the flash peripheral aborts any high voltage operation.

If such catastrophic event occurs (i.e. a power down or reset occurs when an erase/write operation is ongoing), FEE driver shall be able to recover the last valid instance of the blocks stored in the flash, ignoring the possible last block update interrupted by the reset.

During Fee initialization phase, a read operation on the bytes affected by the abort could generate an ECC error.

At start-up the Fee driver will scan the memory in order to restore the cluster and the blocks status. If some data is corrupted, an Ivor exception is thrown during the read operation and Fls driver will manages it.

The driver behaves differently depending on which operation was interrupted:

- a cluster swap is ongoing,
- a block update is ongoing,
- an immediate block update is ongoing.

4.5 Cluster Swap

The cluster swap is a way how unlimited erase/write cycles required by AUTOSAR are implemented.

A cluster swap occurs in the following cases:

- If the active cluster has not enough free space to host a new block.
- If some FEE block header is invalid.
- If a flash job has failed during FEE initialization stage.
- If some FEE block number is invalid.
- If some FEE block header doesn't match FEE configuration.
- If the last header is corrupted.
- when trying to read/write on damaged locations.

The swap consists of the following steps (stages):

1. Erase the next cluster (ERASING stage).
2. Write the first part of the cluster header (16 byte: incremented ClrID, StartAddress, ClusterSize, Checksum) (FORMATTING stage).
3. Copy the last instances of all blocks except the block that generated the cluster swap (header, data and status) (COPYING(n-1) stage). If FEE_BlocksAlwaysAvailable == STD_ON, all blocks are copied.
4. Write the second part of cluster header (16 bytes: valid/invalid flag) (ACTIVE stage);
5. Write the block that generated the cluster swap (header, data and status) (COPYING(n) stage) to the newly allocated cluster.

Table 4-5. Cluster Swap Stages

CLUSTER	STAGE 1	STAGE 2	STAGE 3	STAGE 4	STAGE 5
ID 0001	ACTIVE	ACTIVE	ACTIVE	OLD	OLD
ID 0002	ERASING	FORMATTING	COPYING(n-1)	ACTIVE	COPYING(n)

A system reset happens during STAGE 1

Since an erase operation may have been interrupted, the next cluster could be affected by ECC error.

A system reset happens during STAGE 2 and STAGE 4.

Since a program operation may have been interrupted, the next cluster header could be affected by ECC error.

A system reset happens during STAGE 3

Since a program operation may have been interrupted, the next cluster area could be affected by ECC error.

A system reset happens during STAGE 5.

Since a program operation may have been interrupted, the next cluster area could be affected by ECC error.

The active cluster is the one with ID 0002.

If the cluster swap process is broken in any of the preceding stages, the following happens during next startup:

- The application should run the FEE initialization phase by calling Fee_Init and then repeatedly Fee_MainFunction and Fls_MainFunction until the driver is in the idle state.
- During this phase the active cluster is recognized and it is not affected by ECC error.
- Only the block that caused cluster swap is lost. If the application writes any block, a new cluster swap is initiated. This will erase the cluster again and remove the ECC error wherever it is.

4.6 Block Update

During normal execution (without any catastrophic event) the consistency of data block is assured by the order in which the block fields are written to the memory.

The block updating process consists of the following steps:

- writing the BlkId, StartAddress Length and CheckSum fields in the header area;
- writing Data in the data area;
- updating the Status of block from INCONSISTENT to VALID.

In case of a catastrophic event during block updating after powering the system again a Fee Initialization phase should be re-executed after power up the system.

During this phase:

- the active cluster will be selected;
- the header blocks zone will be scanned in order to restore the status of blocks before the power down.

If more instances of the same block are present in the cluster, only the instance with highest address is kept as valid.

If an ECC error occurs, Fee_Init considers the block affected invalid, and keeps the previous instance. If there is no block instance, the block is considered invalid either.

4.7 Immediate Block Update

During normal execution (without a catastrophic event) immediate data blocks are updated in two steps:

1. **first phase:** The header is written by calling Fee_EraseImmediateBlock (and appropriate amount of FeeMainFunction/Fls Mainfunction calls)
 - the BlkId, StartAddress, Length and CheckSum fields in the header area are written, and
 - space for data is reserved.

Note: This step is mandatory.

2. **second phase** Write the actual data by calling the Fee_Write function:
 - the data portion is written, and
 - the status of block is updated from MEMIF_BLOCK_INCONSISTENT to FEE_BLOCK_VALID by writing the valid flag.

Important note: If a power drop occurs during immediate block write operation, the next Fee initialization phase may consider the space reserved for this immediate block as correct even if there are already some data written (there is no blank check). It is responsibility of the application to use the Fee_EraseImmediateBlock function before each immediate block write operation.

Chapter 5

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the FEE Driver. The most of the parameters are described below.

5.1 Configuration elements of Fee

Included forms :

- IMPLEMENTATION_CONFIG_VARIANT
- FeeGeneral
- CommonPublishedInformation
- FeePublishedInformation
- FeeClusterGroup
- FeeBlockConfiguration

Table 5-1. Revision table

Revision	Date
4.1.0	2010-12-03

5.2 Form IMPLEMENTATION_CONFIG_VARIANT

VariantPreCompile: Precompile configuration parameters.

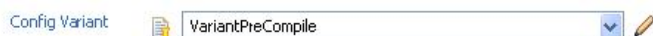


Figure 5-1. Tresos Plugin snapshot for IMPLEMENTATION_CONFIG_VARIANT form.

Table 5-2. Attribute IMPLEMENTATION_CONFIG_VARIANT detailed description

Property	Value
Label	Config Variant
Default	VariantPreCompile

5.3 Form FeeGeneral

Container for general parameters. These parameters are not specific to a block.

Fee General

Fee General

Fee Development Error Detect ☒

Fee Index (0 -> 255) 0

Fee Nvm Job End Notification NULL_PTR

Fee Nvm Job Error Notification NULL_PTR

Fee Polling Mode ☐

Fee Virtual Page Size (16 -> 65535) 16

Fee Data Buffer Size (0 -> 65535) 48

Fee Block Always Available ☐

Fee Version Info Api ☒

Figure 5-2. Tresos Plugin snapshot for FeeGeneral form.

5.3.1 FeeDevErrorDetect (FeeGeneral)

Pre-processor switch to enable and disable development error detection.

true: Development error detection enabled.

false: Development error detection disabled.

Table 5-3. Attribute FeeDevErrorDetect (FeeGeneral) detailed description

Property	Value
Label	Fee Development Error Detect

Table continues on the next page...

Table 5-3. Attribute FeeDevErrorDetect (FeeGeneral) detailed description (continued)

Property	Value
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

5.3.2 FeeIndex (FeeGeneral)

Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.

Table 5-4. Attribute FeeIndex (FeeGeneral) detailed description

Property	Value
Label	Fee Index
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=254 >=0

5.3.3 FeeNvmJobEndNotification (FeeGeneral)

Mapped to the job end notification routine provided by the upper layer module (NmJobEndNotification).

Note: Disable the end notification to have it set as NULL_PTR.

Table 5-5. Attribute FeeNvmJobEndNotification (FeeGeneral) detailed description

Property	Value
Label	Fee Nvm Job End Notification
Type	FUNCTION-NAME
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	JobEndNotification
Enable	false

5.3.4 FeeNvmJobErrorNotification (FeeGeneral)

Mapped to the job error notification routine provided by the upper layer module (NvM_JobErrorNotification).

Note: Disable the error notification to have it set as NULL_PTR.

Table 5-6. Attribute FeeNvmJobErrorNotification (FeeGeneral) detailed description

Property	Value
Label	Fee Nvm Job Error Notification
Type	FUNCTION-NAME
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	JobErrorNotification
Enable	false

5.3.5 FeePollingMode (FeeGeneral)

Pre-processor switch to enable and disable the polling mode for this module

Note

This parameter is not utilized in this BSW module implementation

Table 5-7. Attribute FeePollingMode (FeeGeneral) detailed description

Property	Value
Label	Fee Polling Mode
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

5.3.6 FeeSetModeSupported (FeeGeneral)

Compiler switch to enable/disable the SetMode functionality of the FEE module.

true: SetMode functionality supported / code present,

false: SetMode functionality not supported / code not present.

Note

This configuration setting has to be consistent with that of all underlying flash device drivers (configuration parameter FlsSetModeApi).

Table 5-8. Attribute FeeSetModeSupported (FeeGeneral) detailed description

Property	Value
Label	Fee Set Mode Supported
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

5.3.7 FeeVersionInfoApi (FeeGeneral)

Pre-processor switch to enable / disable the API to read out the modules version information.

true: Version info API enabled.

false: Version info API disabled.

Table 5-9. Attribute FeeVersionInfoApi (FeeGeneral) detailed description

Property	Value
Label	Fee Version Info Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

5.3.8 FeeVirtualPageSize (FeeGeneral)

The size in bytes to which logical blocks shall be aligned.

Table 5-10. Attribute FeeVirtualPageSize (FeeGeneral) detailed description

Property	Value
Label	Fee Virtual Page Size
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	16
Invalid	Range <=65535 >=8

5.3.9 FeeDataBufferSize (FeeGeneral)

Vendor specific: Size of the data buffer in bytes.

The data buffer is used to buffer data when Fee is copying data from one cluster to another and when Fee is reading the block header information on startup.

Size of the data buffer affects number of Fls_MainFunction cycles. Bigger data buffer improves performance of the Fee cluster management operations and speeds up the startup phase as Fee can read more data in one cycle of Fls_MainFunction

Note

FeeDataBufferSize must be equal or greater than FEE_CLUSTER_OVERHEAD. Where FEE_CLUSTER_OVERHEAD is management overhead per logical cluster in bytes and can be calculated using the following formula: $\text{ceiling}(16 / \text{FEE_VIRTUAL_PAGE_SIZE} + 2) * \text{FEE_VIRTUAL_PAGE_SIZE}$

Table 5-11. Attribute FeeDataBufferSize (FeeGeneral) detailed description

Property	Value
Label	Fee Data Buffer Size
Type	INTEGER
Origin	Custom
Symbolic Name	false
Invalid	Range >=0 <=65535

5.3.10 FeeBlockAlwaysAvailable (FeeGeneral)

Vendor specific: According to the AUTOSAR requirements, when the write operation starts, corresponding block is marked as inconsistent. It is marked as consistent after successful write. It means that when write operation is interrupted (cancel, reset) application cannot access the block data anymore.

Enabling this parameter allows to set the behavior against the AUTOSAR requirements. In this case, the last valid information is always provided. Valid information means:

- FEE_BLOCK_INVALID if the block was invalidated or the block is not present in the cluster at all.
- FEE_BLOCK_VALID in case the previous instance of the block exists and was not invalidated.

Table 5-12. Attribute FeeBlockAlwaysAvailable (FeeGeneral) detailed description

Property	Value
Label	Fee Block Always Available
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

5.4 Form CommonPublishedInformation

CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

▼ CommonPublishedInformation

Name

CommonPublishedInformation

AUTOSAR Major Version

4

AUTOSAR Minor Version

0

AUTOSAR Revision Version

3

Numeric Module ID

21

Software Major Version

1

Software Minor Version

0

Software Patch Version

1

Vendor Api Infix

Vendor ID

43

Figure 5-3. Tresos Plugin snapshot for CommonPublishedInformation form.

5.4.1 ArReleaseMajorVersion (CommonPublishedInformation)

AUTOSAR Major Version

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 5-13. Attribute ArReleaseMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Major Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	4
Invalid	Range >=4 <=4

5.4.2 ArReleaseMinorVersion (CommonPublishedInformation)

AUTOSAR Minor Version

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 5-14. Attribute ArReleaseMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range ≥ 0 ≤ 0

5.4.3 ArReleaseRevisionVersion (CommonPublishedInformation)

AUTOSAR Patch Version

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 5-15. Attribute ArReleaseRevisionVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Revision Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	3
Invalid	Range ≥ 3 ≤ 3

5.4.4 ModuleId (CommonPublishedInformation)

Module ID

Module ID of this module from Module List.

Table 5-16. Attribute ModuleId (CommonPublishedInformation) detailed description

Property	Value
Label	Numeric Module ID
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	21
Invalid	Range >=21 <=21

5.4.5 SwMajorVersion (CommonPublishedInformation)

Software Major Version

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 5-17. Attribute SwMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Major Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range >=1 <=1

5.4.6 SwMinorVersion (CommonPublishedInformation)

Software Minor Version

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 5-18. Attribute SwMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

5.4.7 SwPatchVersion (CommonPublishedInformation)

Software Patch Version

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 5-19. Attribute SwPatchVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Patch Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1
Invalid	Range >=1 <=1

5.4.8 VendorApiInfix (CommonPublishedInformation)

Vendor Api Infix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix><Api name from SWS>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1.

It shall not be used for modules with upper multiplicity =1.

Table 5-20. Attribute VendorApiInfix (CommonPublishedInformation) detailed description

Property	Value
Label	Vendor Api Infix
Type	STRING_LABEL
Origin	Custom
Symbolic Name	false
Default	
Enable	false

5.4.9 VendorId (CommonPublishedInformation)

Vendor ID

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Table 5-21. Attribute VendorId (CommonPublishedInformation) detailed description

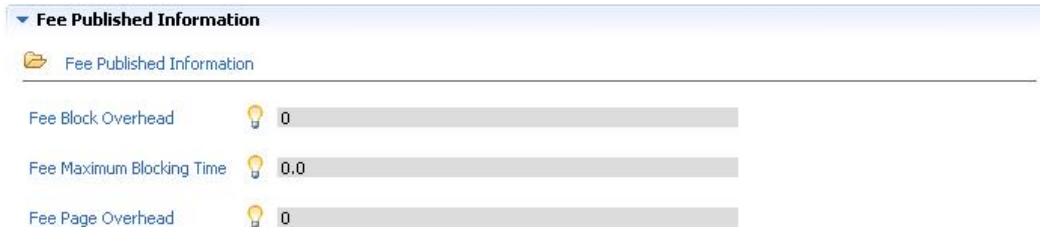
Property	Value
Label	Vendor ID
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	43
Invalid	Range >=43 <=43

5.5 Form FeePublishedInformation

Additional published parameters not covered by CommonPublishedInformation container.

Note

That these parameters do not have any configuration class setting, since they are published information.



The screenshot shows a configuration window titled 'Fee Published Information'. It contains three rows of parameters, each with a lightbulb icon and a text input field:

- Fee Block Overhead: 0
- Fee Maximum Blocking Time: 0.0
- Fee Page Overhead: 0

Figure 5-4. Tresos Plugin snapshot for FeePublishedInformation form.

5.5.1 FeeBlockOverhead (FeePublishedInformation)

Management overhead per logical block in bytes

Note

The logical block management overhead depends on FeeVirtualPageSize and can be calculated using the following formula: $\text{ceiling}(12 / \text{FEE_VIRTUAL_PAGE_SIZE} + 2) * \text{FEE_VIRTUAL_PAGE_SIZE}$

Table 5-22. Attribute FeeBlockOverhead (FeePublishedInformation) detailed description

Property	Value
Label	Fee Block Overhead
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range ≤ 65535 ≥ 0

5.5.2 FeeMaximumBlockingTime (FeePublishedInformation)

The maximum time the FEE module's API routines shall be blocked (delayed) by internal operations.

Note

The maximum blocking time depends on various conditions such as MCU clock configuration, flash device configuration, configuration of underlying flash driver, size of flash sectors used for EEPROM emulation, etc. The maximum blocking time is not constant and can not be calculated using simple formula.

Table 5-23. Attribute FeeMaximumBlockingTime (FeePublishedInformation) detailed description

Property	Value
Label	Fee Maximum Blocking Time
Type	FLOAT_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0.0
Invalid	Range ≤ 1000000.0 ≥ 0.0

5.5.3 FeePageOverhead (FeePublishedInformation)

Management overhead per page in bytes

Note

The page management overhead is 0 bytes

Table 5-24. Attribute FeePageOverhead (FeePublishedInformation) detailed description

Property	Value
Label	Fee Page Overhead
Type	INTEGER_LABEL
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range ≤ 65535 ≥ 0

5.6 Form FeeClusterGroup

Vendor specific: Configuration of cluster group specific parameters for the Flash EEPROM Emulation module.

Included forms :

- [Form FeeCluster](#)

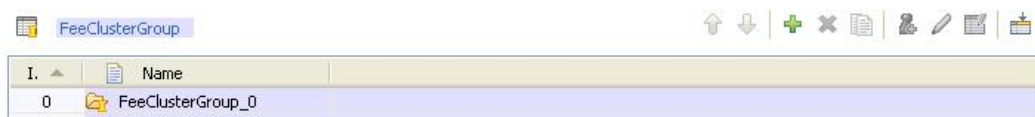


Figure 5-5. Tressos Plugin snapshot for FeeClusterGroup form.

5.6.1 Form FeeCluster

Vendor specific: Configuration of cluster specific parameters for the Flash EEPROM Emulation module.

Is included by form : [Form FeeClusterGroup](#)

Included forms :

- [Form FeeSector](#)

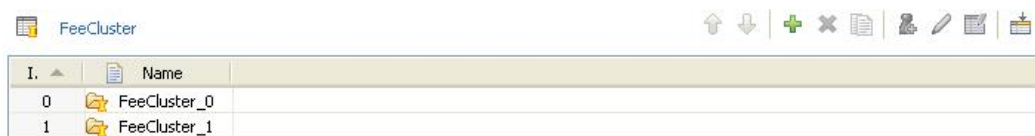
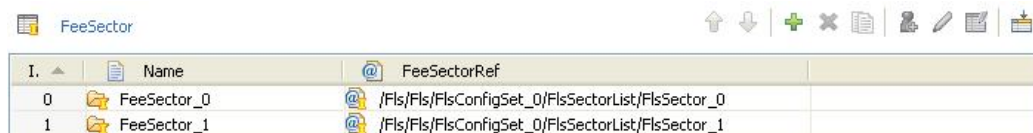


Figure 5-6. Tressos Plugin snapshot for FeeCluster form.

5.6.1.1 Form FeeSector

Vendor specific: Configuration of sector specific parameters for the Flash EEPROM Emulation module.

Is included by form : [Form FeeCluster](#)



I.	Name	FeeSectorRef
0	FeeSector_0	/Fls/Fls/FlsConfigSet_0/FlsSectorList/FlsSector_0
1	FeeSector_1	/Fls/Fls/FlsConfigSet_0/FlsSectorList/FlsSector_1

Figure 5-7. Tresos Plugin snapshot for FeeSector form.

5.6.1.1.1 FeeSectorRef (FeeSector)

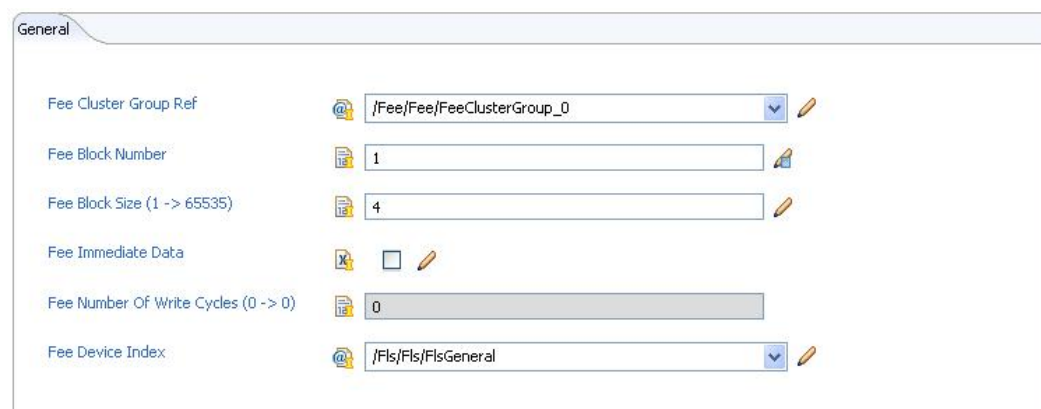
Vendor specific: Reference to a logical Fls sector the Fee cluster consist of.

Table 5-25. Attribute FeeSectorRef (FeeSector) detailed description

Property	Value
Label	Fee Sector Ref
Type	REFERENCE
Origin	Custom

5.7 Form FeeBlockConfiguration

Configuration of block specific parameters for the Flash EEPROM Emulation module.



Fee Cluster Group Ref	/Fee/Fee/FeeClusterGroup_0
Fee Block Number	1
Fee Block Size (1 -> 65535)	4
Fee Immediate Data	☐
Fee Number Of Write Cycles (0 -> 0)	0
Fee Device Index	/Fls/Fls/FlsGeneral

Figure 5-8. Tresos Plugin snapshot for FeeBlockConfiguration form.

5.7.1 FeeBlockNumber (FeeBlockConfiguration)

Block identifier (handle).

0x0000 and 0xFFFF shall not be used for block numbers (see FEE006).

Range:

min = $2^{\text{NVM_DATA_SELECTION_BITS}}$

max = $0\text{xFFFF} - 2^{\text{NVM_DATA_SELECTION_BITS}}$

Note

: Depending on the number of bits set aside for dataset selection several other block numbers shall also be left out to ease implementation.

Table 5-26. Attribute FeeBlockNumber (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Block Number
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	true
Invalid	Range ≤ 65534 ≥ 1

5.7.2 FeeBlockSize (FeeBlockConfiguration)

Size of a logical block in bytes.

Table 5-27. Attribute FeeBlockSize (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Block Size
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	1
Invalid	Range ≤ 65535 ≥ 1

5.7.3 FeeImmediateData (FeeBlockConfiguration)

Marker for high priority data.

true: Block contains immediate data.

false: Block does not contain immediate data.

Table 5-28. Attribute FeeImmediateData (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Immediate Data
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

5.7.4 FeeNumberOfWriteCycles (FeeBlockConfiguration)

Vendor specific: Number of write cycles required for this block.

Table 5-29. Attribute FeeNumberOfWriteCycles (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Number Of Write Cycles
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range <=4294967295 >=0

5.7.5 FeeClusterGroupRef (FeeBlockConfiguration)

Vendor specific: Reference to the Fee cluster group which the Fee block belongs to. In other words, FeeClusterGroupRef assigns the Fee block to particular Fee cluster group.

Table 5-30. Attribute FeeClusterGroupRef (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Cluster Group Ref
Type	REFERENCE
Origin	Custom

5.7.6 FeeDeviceIndex (FeeBlockConfiguration)

Device index (handle).

Range: 0 .. 254 (0xFF reserved for broadcast call to GetStatus function).

Table 5-31. Attribute FeeDeviceIndex (FeeBlockConfiguration) detailed description

Property	Value
Label	Fee Device Index
Type	SYMBOLIC-NAME-REFERENCE
Origin	AUTOSAR_ECUC

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13FEEASR4.0 Rev0003R1.0.1
Revision 1.2